



**UNIVERSIDADE FEDERAL
DE MATO GROSSO**

UNIVERSIDADE FEDERAL DE MATO GROSSO (UFMT)
INSTITUTO DE CIÊNCIAS EXATAS DA TERRA (ICET)
PROJETO DE SOFTWARE

GABRIEL PIVETTA LOSS, JOÃO MARCELLO GALDINO PEREIRA, VINICIUS DE
SOUZA MENDES, PEDRO JOÃO PAULINO FRANZ E SAMUEL CAMPOS MARTINS

MODELAGEM DE SOFTWARE E ENGENHARIA REVERSA
Análise do Sistema Batalha Naval

BARRA DO GARÇAS - MT
2026

UNIVERSIDADE FEDERAL DE MATO GROSSO (UFMT)
GABRIEL PIVETTA LOSS, JOÃO MARCELLO GALDINO PEREIRA, VINICIUS DE
SOUZA MENDES, PEDRO JOÃO PAULINO FRANZ E SAMUEL CAMPOS MARTINS

MODELAGEM DE SOFTWARE E ENGENHARIA REVERSA
Análise do Sistema Batalha Naval

Trabalho apresentado como requisito à obtenção de uma das
notas parciais da disciplina de Projeto de Software do curso
de Ciência da Computação da Universidade Federal de Mato
Grosso.

Thiago Pereira Silva

Sumário

	Páginas
1 Análise geral do projeto batalha naval	4
1.1 Arquitetura geral da aplicação	4
1.2 Organização dos pacotes e módulos	5
1.3 Classes e suas responsabilidades	5
1.3.1 Relacionamento entre classes	6
1.4 Padrões de projeto utilizados	6
1.5 Principais funcionalidades	6
1.6 Fluxo de execução	7
2 Propostas de Solução e Melhorias Arquiteturais	7
2.1 Separação em View, Control e Model	7
2.1.1 Camada View	8
2.1.2 Camada Control	8
2.1.3 Camada Model	9
2.2 Representação da Partida	9
2.3 Criação da Classe Tabuleiro	10
2.4 Criação da Classe Celula	10
2.5 Representação das Embarcações	10
2.6 Controle do Adversário	11
2.7 Melhoria no Ranking	11
2.8 Novo Cronômetro	11
2.9 Novo Fluxo de Funcionamento	11
2.10 Problemas Resolvidos pela Proposta	12
2.11 Resumo da Arquitetura Proposta	12

1.2 Organização dos pacotes e módulos

O projeto apresenta uma organização simples, concentrando praticamente todas as classes no pacote `batalhanaval`. Embora exista separação física entre o código-fonte (`src`) e os recursos gráficos (imagens), não há divisão lógica em módulos ou camadas arquiteturais.

1.3 Classes e suas responsabilidades

Apesar do alto acoplamento, a aplicação é dividida em dez classes principais, distribuídas entre a interface gráfica e o suporte à persistência:

- `BatalhaNaval`: Ponto de entrada da aplicação, responsável apenas por conter o método `main` e instanciar a primeira janela.
- `StartJanela`: Interface inicial que coleta o nome do jogador, valida o preenchimento e gerencia a transição para a etapa de configuração.
- `SetupJanela`: Centraliza a lógica de configuração da partida. É responsável por renderizar a interface de posicionamento, processar a inserção manual de navios e executar o algoritmo de geração aleatória de tabuleiros.
- `GameJanela`: O núcleo do sistema. Acumula as responsabilidades de renderizar os tabuleiros (jogador e computador), gerenciar a alternância de turnos, validar e aplicar os diferentes tipos de disparos, controlar o fluxo de vitória/derrota e disparar o salvamento do ranking.
- `EndJanela`: Exibe o resultado final da partida (vitória ou derrota), renderiza a tabela de ranking e oferece opções de reinício ou encerramento do programa.
- `GameButton`: Extensão customizada da classe `JButton` do Swing. Funciona de forma híbrida: além do comportamento visual, armazena dados de modelo (coordenadas de linha e coluna, além do tipo de ocupação da célula).
- `Cronometro`: Classe que estende `Thread`, responsável pela contagem incremental do tempo de jogo em segundo plano e atualização direta do componente textual na interface gráfica.
- `Tempos`: Gerenciador do ranking em memória, controlando a inserção de novos registros e delegando a leitura e escrita física de dados.
- `Info`: Objeto de transferência de dados (DTO) que encapsula as informações de um registro do ranking (nome, tempo total e data).
- `Arquivo`: Classe utilitária com métodos estáticos responsáveis pela serialização e desserialização física do arquivo binário de ranking (`tempos.dat`).

- **CompareInfo:** Implementa a interface `Comparator`, definindo o critério de ordenação do ranking com base no menor tempo de partida.

1.3.1 Relacionamento entre classes

Os relacionamentos expõem a arquitetura monolítica e a passagem de responsabilidade linear em forma de "cadeia de janelas". Os principais vínculos identificados são:

- **Generalização e Realização:** As classes `StartJanela`, `SetupJanela`, `GameJanela` e `EndJanela` estendem `JFrame` e implementam a interface `ActionListener`, vinculando o comportamento de negócio diretamente aos eventos de clique do Swing. A classe `GameButton` estende `JButton`, e `Cronometro` estende `Thread`.
- **Composição:** Existe uma forte relação de composição entre as telas de jogo (`SetupJanela` e `GameJanela`) e o componente `GameButton`, materializada em matrizes bidimensionais de 10×10 elementos para representar os tabuleiros.
- **Dependência e Associação:** O fluxo de telas ocorre por dependência direta, onde uma janela instancia a sucessora (ex: `StartJanela` instancia `SetupJanela`). No subsistema de ranking, `Tempos` mantém uma associação (coleção) com objetos `Info`, além de depender de `CompareInfo` para ordenação e dos métodos estáticos de `Arquivo` para persistência.

1.4 Padrões de projeto utilizados

Durante a análise não foi identificado o uso explícito de padrões de projeto tradicionais, como MVC, Observer, Factory ou Strategy. A aplicação foi desenvolvida seguindo um fluxo procedural orientado a eventos do Swing, concentrando grande parte da lógica de negócio nas próprias classes da interface gráfica.

1.5 Principais funcionalidades

O sistema apresenta um escopo funcional completo para o jogo de Batalha Naval, contendo:

- **Configuração de Perfil e Modo:** Identificação do jogador por nome e escolha do modo de preparação da frota.
- **Posicionamento Inteligente de Frota:** Sistema de posicionamento manual na horizontal com validação de colisão espacial, ou geração automatizada e randômica do tabuleiro do jogador e da inteligência artificial.

- **Mecanismo de Combate:** Inteligência artificial para contra-ataques automáticos e suporte a quatro modalidades de disparo pelo jogador humano (Disparo Simples, Disparo em Cascata, Disparo em Estrela e Disparo Porta-Aviões), cada um afetando uma área geométrica distinta do tabuleiro adversário.
- **Controle de Desempenho e Persistência:** Cronometragem em tempo real da partida e persistência automatizada de records em arquivo binário local, ordenando os melhores jogadores.

1.6 Fluxo de execução

O fluxo dinâmico do sistema é estritamente sequencial e guiado por eventos de interface (Event-Driven). O ciclo de vida de uma execução padrão compreende: 1. Inicialização do sistema pela classe `BatalhaNaval` e abertura da tela de menu (`StartJanela`). 2. Coleta de dados e redirecionamento para a tela de posicionamento (`SetupJanela`). 3. Configuração dos tabuleiros e descarte da tela de setup, abrindo o tabuleiro principal (`GameJanela`). 4. Disparo paralelo da thread `Cronometro` e alternância de turnos de ataque (Jogador e Computador) até que o contador de células de navio de uma das partes seja zerado. 5. Interrupção do tempo, gravação física do resultado caso haja vitória, fechamento da tela de jogo e abertura da tela de encerramento (`EndJanela`) com a tabela de ranking atualizada.

2 Propostas de Solução e Melhorias Arquiteturais

A partir da análise do projeto original, foi possível identificar que a maior parte dos problemas está relacionada à concentração de responsabilidades em poucas classes. No código atual, classes como `SetupJanela` e `GameJanela` acumulam funções de interface gráfica, controle de eventos, regras do jogo, lógica de ataque, controle de fim de partida e integração com ranking.

Como proposta de solução, foi elaborada uma nova organização arquitetural para o sistema, baseada na separação entre **View**, **Control** e **Model**. Essa divisão tem como objetivo separar a interface gráfica das regras do jogo e dos dados do sistema, reduzindo o acoplamento e tornando o projeto mais fácil de manter e evoluir.

2.1 Separação em View, Control e Model

A principal proposta de melhoria é reorganizar o sistema em uma estrutura inspirada no padrão MVC. No projeto original, as telas da aplicação também executam regras importantes do jogo, misturando interface gráfica, controle de eventos e regras da partida. Na proposta corrigida, cada parte passa a ter uma responsabilidade mais clara, separando o sistema em três grupos principais: View, Control e Model.

A camada *View* fica responsável pelas telas e componentes visuais. A camada *Control* atua como mediadora, coordenando as ações principais do jogo e processando as entradas do usuário. Já a camada *Model* representa o coração da aplicação, contendo o domínio central do jogo (tabuleiro, células, navios), a inteligência do adversário e os dados do sistema, como o ranking.

2.1.1 Camada View

No diagrama de melhorias, a camada *View* contém as classes responsáveis pelas telas e componentes visuais da aplicação:

- `StartJanela;`
- `SetupJanela;`
- `GameJanela;`
- `EndJanela;`
- `ComponenteCronometro.`

A classe `StartJanela` continua responsável pela tela inicial, recebendo o nome do jogador e permitindo iniciar uma nova partida. A classe `SetupJanela` representa a tela de configuração do tabuleiro. A classe `GameJanela` passa a ser responsável principalmente por exibir o estado da partida, capturar eventos da interface e encaminhar as ações para o controlador. A classe `EndJanela` exibe o resultado final e o ranking.

Com essa mudança, as janelas deixam de concentrar regras complexas do jogo. Elas passam a apenas exibir informações, receber comandos do usuário e chamar os controladores responsáveis.

2.1.2 Camada Control

A camada *Control* concentra a coordenação e a orquestração das operações principais do jogo. No diagrama, a classe central dessa camada é a `JogoController`.

Entre suas responsabilidades estão:

- iniciar uma nova partida com o nome do jogador e o modo escolhido;
- processar o clique do jogador no tabuleiro inimigo;
- solicitar a execução da jogada do computador;
- verificar as condições de vitória ou derrota;
- ordenar a atualização da tela de jogo quando o estado da partida muda.

Essa organização resolve um dos principais problemas do projeto original: a concentração da lógica de combate dentro de GameJanela. Com o controlador, a tela deixa de decidir sozinha o que acontece no jogo e passa a apenas enviar comandos para a camada de controle, que aplica as regras sobre o modelo.

2.1.3 Camada Model

A camada Model encapsula as regras de negócio, o estado real do jogo e os serviços de apoio. Diferente do código original, onde os dados do tabuleiro ficavam atrelados aos botões da interface gráfica, agora o estado do sistema é isolado e mantido nesta camada.

As principais classes que compõem o domínio central do jogo são:

- Partida: representa o estado geral do jogo, associando os jogadores aos seus tabuleiros;
- Tabuleiro: gerencia a matriz de coordenadas e registra os ataques;
- Celula: representa o estado individual de uma posição no tabuleiro;
- Embarcacao: abstrai os navios e suas propriedades (tamanho, vida e status de destruição).

Além do domínio principal, a camada Model também concentra as classes de apoio relacionadas ao ranking e ao adversário computadorizado:

- IAAdversaria: responsável por calcular e decidir o próximo alvo do computador;
- RankingController: coordena as operações de salvar e recuperar pontuações;
- Ranking: armazena a lista de pontuações e lida com a persistência (leitura e gravação de arquivos);
- JogadorInfo: objeto de transporte (DTO) que encapsula os dados da pontuação.

Com essa separação, o sistema passa a ter uma divisão estritamente aderente às boas práticas: a View exibe a interface e captura ações, a Control orchestra o fluxo da partida e a Model mantém o estado verdadeiro do jogo, a lógica da IA e os dados do ranking.

2.2 Representação da Partida

No diagrama proposto, a classe Partida representa o estado geral de uma partida. Ela armazena os dois tabuleiros principais, o status da partida e o nome do jogador.

A classe Partida possui associação com dois objetos Tabuleiro: um para o jogador e outro para o computador. Também possui operações como trocarTurno() e isFinalizada(), que ajudam a controlar o andamento da partida.

Essa mudança melhora a organização do sistema, pois o estado da partida deixa de ficar espalhado dentro das classes de interface gráfica e passa a ser representado por uma classe própria.

2.3 Criação da Classe Tabuleiro

A classe Tabuleiro foi proposta para representar a estrutura do tabuleiro de cada jogador. No projeto original, o tabuleiro é representado principalmente por matrizes de JButton, fazendo com que o estado do jogo dependa diretamente dos botões da interface.

Na proposta de melhoria, o tabuleiro passa a ser formado por uma matriz de objetos Celula. A classe Tabuleiro fica responsável por registrar tiros, revelar posições e verificar se toda a frota foi destruída.

Essa alteração é importante porque separa o estado do jogo da interface gráfica. Assim, o tabuleiro pode ser testado e manipulado sem depender dos componentes visuais do Swing.

2.4 Criação da Classe Celula

A classe Celula representa uma posição individual do tabuleiro. Ela armazena o estado da célula e, quando necessário, uma referência para a embarcação posicionada naquela posição.

No diagrama, a célula possui informações como:

- status da célula;
- embarcação associada;
- operação para receber impacto;
- verificação se existe embarcação naquela posição.

Essa mudança substitui a ideia de usar diretamente botões como armazenamento de dados. O botão continua existindo na interface, mas o estado real da célula passa a pertencer ao modelo do jogo.

2.5 Representação das Embarcações

Outra melhoria importante é a criação de uma classe abstrata chamada Embarcacao. Essa classe representa uma embarcação genérica do jogo, contendo atributos como nome, tamanho e vida.

A partir dela, são especializadas classes específicas para cada tipo de embarcação:

- PortaAvioes;
- NavioEscolta;
- Submarino;
- Caca.

Essa estrutura melhora a representação dos navios dentro do sistema. Em vez de trabalhar apenas com textos ou contadores espalhados pela interface, cada embarcação passa a ter seu próprio estado e comportamento, como receber dano e verificar se foi afundada.

2.6 Controle do Adversário

O diagrama também propõe a classe `IAAdversaria`, responsável por calcular o próximo alvo do computador. No projeto original, a lógica do ataque do computador fica misturada dentro da classe `GameJanela`.

Com a criação da `IAAdversaria`, a decisão do computador passa a ficar isolada em uma classe própria. Isso facilita a manutenção e permite melhorar a inteligência do adversário futuramente sem alterar diretamente a tela do jogo.

Essa separação também torna possível criar diferentes estratégias para o computador, como ataques aleatórios, ataques mais inteligentes ou níveis de dificuldade.

2.7 Melhoria no Ranking

No diagrama de melhorias, o ranking passa a ser tratado por classes específicas: `RankingController`, `Ranking` e `JogadorInfo`.

A classe `JogadorInfo` representa os dados de uma pontuação, como nome do jogador, tempo total e data da partida. A classe `Ranking` armazena a lista de pontuações e fica responsável por carregar e salvar os dados. Já a classe `RankingController` coordena as operações de adicionar pontuação e obter os melhores resultados.

Essa separação melhora a organização do sistema, pois a lógica do ranking deixa de ficar acoplada diretamente às telas finais do jogo. A `EndJanela`, por exemplo, passa apenas a solicitar os dados do ranking para exibição.

2.8 Novo Cronômetro

O diagrama propõe a substituição do cronômetro atual por um `ComponenteCronometro`. Esse componente utiliza um `javax.swing.Timer`, sendo mais adequado para aplicações Swing do que atualizar a interface diretamente por meio de uma thread separada.

Com essa melhoria, o controle de tempo fica encapsulado em um componente próprio, contendo operações como iniciar, parar e retornar o tempo total em segundos.

Essa mudança melhora a segurança da atualização da interface e reduz o acoplamento entre o cronômetro e a classe principal do jogo.

2.9 Novo Fluxo de Funcionamento

Com a arquitetura proposta, o fluxo do sistema passa a funcionar de forma mais organizada.

Primeiro, a `StartJanela` recebe o nome do jogador e solicita ao `JogoController` o início de uma nova partida. Em seguida, a `SetupJanela` permite configurar o tabuleiro e envia as ações para o controlador. Durante a partida, a `GameJanela` apenas captura os eventos de clique e repassa para o `JogoController`, que processa o tiro, atualiza a `Partida`, consulta a `IAAdversaria` quando necessário e manda a interface atualizar a visualização.

Ao final da partida, a `GameJanela` solicita a exibição da `EndJanela`. A tela final consulta o `RankingController`, que acessa o `Ranking` e retorna os melhores resultados para exibição.

Dessa forma, cada parte do sistema passa a ter uma função mais clara.

2.10 Problemas Resolvidos pela Proposta

A proposta representada no diagrama resolve os principais problemas identificados no projeto original:

- reduz a concentração de responsabilidades em `GameJanela`;
- separa a interface gráfica das regras do jogo;
- remove a dependência direta entre estado do tabuleiro e botões da interface;
- organiza melhor o estado da partida;
- separa a lógica do adversário em uma classe própria;
- melhora a estrutura do ranking;
- facilita futuras manutenções e expansões do sistema.

2.11 Resumo da Arquitetura Proposta

De forma geral, o diagrama de melhorias propõe a seguinte reorganização baseada no padrão MVC:

- **View:** responsável estritamente pelas telas, componentes visuais e captura de eventos do usuário;
- **Control:** atua como orquestrador, responsável por receber os eventos da `View`, processar as ações da partida e repassá-las para atualizar o modelo;
- **Model:** concentra o domínio principal do jogo (`Partida`, `Tabuleiro`, `Celula` e `Embarcacao`), o gerenciamento de dados de **Ranking** (`RankingController`, `Ranking` e `JogadorInfo`) e as regras do adversário virtual (`IAAdversaria`).

Essa proposta torna o sistema mais modular, reduz drasticamente o acoplamento entre interface e regras de negócio, melhora a coesão das classes e deixa o projeto muito mais preparado para futuras manutenções e evoluções.

